

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 4

Базы данных

Выполнил

Добкес Михаил Константинович,

группа Р3106

Проверил

Преподаватель Мартин Райла

Санкт-Петербург, 2026

Содержание

Постановка задания.....	3
Выполнение работы.....	4
Запрос 1.....	4
Условие.....	4
Запрос.....	4
Индексы.....	5
План выполнения.....	6
EXPLAIN ANALYZE.....	7
Изменения при добавлении индексов.....	7
EXPLAIN ANALYZE с индексами.....	7
Запрос 2.....	8
Условие.....	8
Запрос.....	8
Индексы.....	9
План выполнения.....	10
EXPLAIN ANALYZE.....	11
Изменения при добавлении индексов.....	11
EXPLAIN ANALYZE с индексами.....	12

Постановка задания

Введите вариант:

Внимание! У разных вариантов разный текст задания!

Составить запросы на языке SQL (пункты 1-2).

Для каждого запроса предложить индексы, добавление которых уменьшит время выполнения запроса (указать таблицы/атрибуты, для которых нужно добавить индексы, написать тип индекса; объяснить, почему добавление индекса будет полезным для данного запроса).

Для запросов 1-2 необходимо составить возможные планы выполнения запросов. Планы составляются на основании предположения, что в таблицах отсутствуют индексы. Из составленных планов необходимо выбрать оптимальный и объяснить свой выбор. Изменяются ли планы при добавлении индекса и как?

Для запросов 1-2 необходимо добавить в отчет вывод команды EXPLAIN ANALYZE [запрос]

Подробные ответы на все вышеперечисленные вопросы должны присутствовать в отчете (планы выполнения запросов должны быть нарисованы, ответы на вопросы - представлены в текстовом виде).

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ИД, Н_СЕССИЯ.ДАТА.

Фильтры (AND):

a) Н_ЛЮДИ.ФАМИЛИЯ < Афанасьев.

b) Н_СЕССИЯ.ИД < 32199.

c) Н_СЕССИЯ.ИД < 27640.

Вид соединения: INNER JOIN.

2. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.ЧЛВК_ИД, Н_УЧЕНИКИ.ГРУППА.

Фильтры: (AND)

a) Н_ЛЮДИ.ФАМИЛИЯ > Иванов.

b) Н_ОБУЧЕНИЯ.НЗК = 001000.

Вид соединения: RIGHT JOIN.

Выполнение работы

Запрос 1

Условие

Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_СЕССИЯ.

Вывести атрибуты: Н_ЛЮДИ.ИД, Н_СЕССИЯ.ДАТА.

Фильтры (AND):

а) Н_ЛЮДИ.ФАМИЛИЯ < Афанасьев.

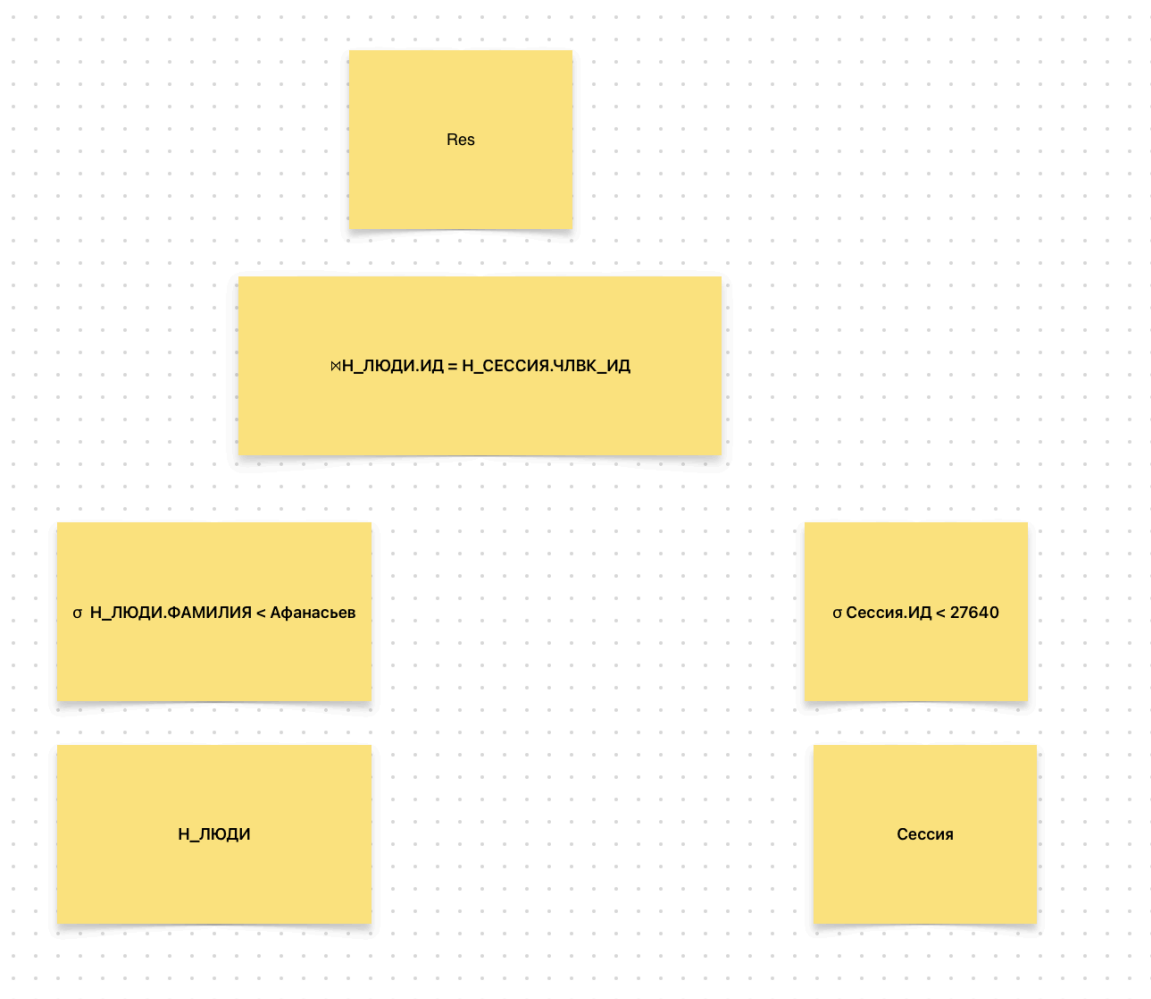
б) Н_СЕССИЯ.ИД < 32199.

с) Н_СЕССИЯ.ИД < 27640.

Вид соединения: INNER JOIN.

Запрос

```
SELECT "Н_люди"."ИД", "Н_сессия"."ДАТА" FROM "Н_люди"  
  
INNER JOIN "Н_сессия"  
  
ON "Н_люди"."ИД" = "Н_сессия"."ЧЛВК_ИД"  
  
WHERE "Н_люди"."ФАМИЛИЯ" < 'Афанасьев'  
  
AND "Н_сессия"."ИД" < 32199  
  
AND "Н_сессия"."ИД" < 27640;
```



Индексы

Для начала рассмотрим порядок проверки индексов

```
WHERE "Н_СЕССИЯ"."ИД" < 27640
```

```
AND "Н_ЛЮДИ"."ФАМИЛИЯ" < 'Афанасьев'
```

```
JOIN ON "Н_ЛЮДИ"."ИД" = "Н_СЕССИЯ"."ЧЛВК_ИД"
```

```
SELECT "ИД", "ДАТА"
```

Индекс 1

Таблица: Н_СЕССИЯ

Атрибут: ИД, ЧЛВК_ИД, ДАТА

Тип индекса: B-Tree

Обоснование: Условие ИД < 27640 стоит первым для отсечения диапазона (условие < 32199 проигнорируется как избыточное). ЧЛВК_ИД идёт вторым для быстрого соединения, а ДАТА добавлена последней, чтобы покрыть столбец из SELECT. Это позволяет выполнить Index-Only Scan, не обращаясь к куче таблицы.

Индекс 2

Таблица: Н_ЛЮДИ

Атрибут: ФАМИЛИЯ, ИД

Тип индекса: B-Tree

Обоснование: Диапазонный фильтр ФАМИЛИЯ < 'Афанасьев' идёт первым. ИД добавлен вторым, так как используется в условии JOIN и возвращается в

SELECT. После фильтрации по фамилии планировщик сразу получает ИД из индекса, избегая случайных чтений страниц основной таблицы.

План выполнения

Возможные варианты:

1. Nested Loop + Seq Scan → полный перебор Н_СЕССИЯ и Н_ЛЮДИ для каждой пары.
2. Merge Join + Sort → сортировка обеих таблиц по ИД/ЧЛВК_ИД перед соединением.
3. Hash Join → хеширование отфильтрованной части Н_СЕССИЯ, затем сканирование Н_ЛЮДИ.

Оптимальный план: Hash Join.

Почему: Условие $ИД < 27640$ отсекает значительную часть строк ещё на этапе сканирования. Хеш-таблица строится по уменьшенному набору данных, что делает этот алгоритм самым быстрым при отсутствии индексов. Merge Join проигрывает из-за обязательной сортировки, а Nested Loop создаёт избыточные повторные чтения.

EXPLAIN ANALYZE

```
Hash Join (cost=166.70..308.23 rows=127 width=12) (actual time=1.069..1.951 rows=81 loops=1)
" Hash Cond: ("Н_СЕССИЯ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
" -> Seq Scan on "Н_СЕССИЯ" (cost=0.00..127.28 rows=3461 width=12) (actual time=0.009..0.647 rows=3460 loops=1)
" Filter: (("ИД" < 32199) AND ("ИД" < 27640))
" Rows Removed by Filter: 292
-> Hash (cost=163.97..163.97 rows=218 width=4) (actual time=1.003..1.003 rows=216 loops=1)
" Buckets: 1024 Batches: 1 Memory Usage: 16kB
" -> Seq Scan on "Н_ЛЮДИ" (cost=0.00..163.97 rows=218 width=4) (actual time=0.006..0.971 rows=216 loops=1)
" Filter: (("ФАМИЛИЯ")::text < 'Афанасьев')::text)
" Rows Removed by Filter: 4903
Planning Time: 0.113 ms
Execution Time: 1.975 ms
```

Изменения при добавлении индексов

- Seq Scan по Н_ЛЮДИ → Index Scan по ("ФАМИЛИЯ", "ИД").
Диапазон < 'Афанасьев' проходится напрямую.

EXPLAIN ANALYZE с индексами

```
Hash Join (cost=22.82..164.35 rows=127 width=12) (actual time=0.137..0.780 rows=81 loops=1)
" Hash Cond: ("Н_СЕССИЯ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
" -> Seq Scan on "Н_СЕССИЯ" (cost=0.00..127.28 rows=3461 width=12) (actual time=0.005..0.437 rows=3460 loops=1)
" Filter: (("ИД" < 32199) AND ("ИД" < 27640))
" Rows Removed by Filter: 292
-> Hash (cost=20.10..20.10 rows=218 width=4) (actual time=0.101..0.101 rows=216 loops=1)
" Buckets: 1024 Batches: 1 Memory Usage: 16kB
" -> Index Only Scan using idx_q2_people_surname_id on "Н_ЛЮДИ" (cost=0.28..20.10 rows=218 width=4) (actual time=0.043..0.043 rows=216 loops=1)
" Index Cond: ("ФАМИЛИЯ" < 'Афанасьев')::text)
" Heap Fetches: 2
Planning Time: 0.277 ms
Execution Time: 0.797 ms
```

Запрос 2

Условие

1. Сделать запрос для получения атрибутов из указанных таблиц, применив фильтры по указанным условиям:

Таблицы: Н_ЛЮДИ, Н_ОБУЧЕНИЯ, Н_УЧЕНИКИ.

Вывести атрибуты: Н_ЛЮДИ.ФАМИЛИЯ, Н_ОБУЧЕНИЯ.ЧЛВК_ИД,
Н_УЧЕНИКИ.ГРУППА.

Фильтры: (AND)

а) Н_ЛЮДИ.ФАМИЛИЯ > Иванов.

б) Н_ОБУЧЕНИЯ.НЗК = 001000.

Вид соединения: RIGHT JOIN.

Запрос

```
SELECT

  "Н_люди"."ФАМИЛИЯ",

  "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД",

  "Н_УЧЕНИКИ"."ГРУППА"

FROM "Н_люди"

RIGHT JOIN "Н_ОБУЧЕНИЯ"

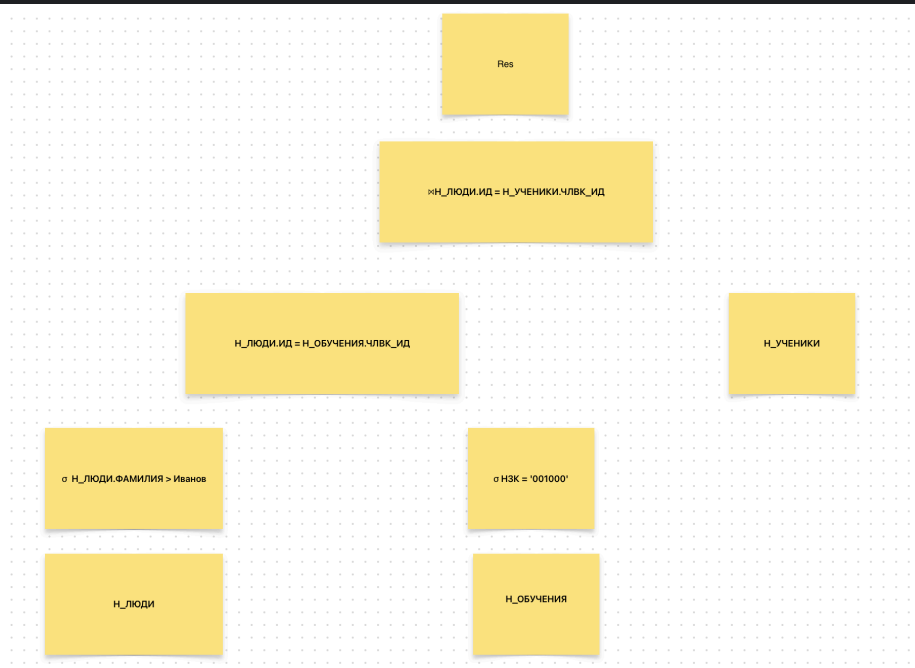
  ON "Н_люди"."ИД" = "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД"

RIGHT JOIN "Н_УЧЕНИКИ"

  ON "Н_люди"."ИД" = "Н_УЧЕНИКИ"."ЧЛВК_ИД"

WHERE "Н_люди"."ФАМИЛИЯ" > 'Иванов'

AND "Н_ОБУЧЕНИЯ"."НЗК" = '001000';
```



Индексы

Для начала рассмотрим порядок проверки индексов

```
WHERE "Н_ОБУЧЕНИЯ"."НЗК" = '001000'
```

```
AND "Н_ЛЮДИ"."ФАМИЛИЯ" > 'Иванов'
```

```
JOIN ON "Н_ЛЮДИ"."ИД" = "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД"
```

```
JOIN ON "Н_ЛЮДИ"."ИД" = "Н_УЧЕНИКИ"."ЧЛВК_ИД"
```

```
SELECT "ФАМИЛИЯ", "ЧЛВК_ИД", "ГРУППА"
```

Индекс 1

Таблица: Н_ОБУЧЕНИЯ

Атрибут: НЗК, ЧЛВК_ИД

Тип индекса: B-Tree

Обоснование: Условие НЗК = '001000' стоит первым для мгновенного отсева строк. ЧЛВК_ИД идёт вторым, что позволяет планировщику сразу получать значения для JOIN.

Индекс 2

Таблица: Н_ЛЮДИ

Атрибут: ФАМИЛИЯ, ИД

Тип индекса: B-Tree

Обоснование: Диапазонный фильтр ФАМИЛИЯ > 'Иванов' идёт первым. ИД добавлен вторым, чтобы индекс покрывал условие JOIN. После фильтрации

по фамилии PostgreSQL сразу получает ИД из индекса, избегая случайного чтения страниц таблицы.

Индекс 3

Таблица: Н_УЧЕНИКИ

Атрибут: ЧЛВК_ИД, ГРУППА

Тип индекса: B-Tree

Обоснование: Покрывающий составной индекс. ЧЛВК_ИД используется для точного поиска при JOIN, а ГРУППА идёт последней. Это позволяет выполнить второй JOIN и вернуть результат полностью из индекса, не читая саму таблицу Н_УЧЕНИКИ.

План выполнения

Возможные варианты (при полном сканировании таблиц Seq Scan):

1. Nested Loop + Seq Scan → для каждой отфильтрованной строки из Н_ОБУЧЕНИЯ запускается полный перебор Н_ЛЮДИ и Н_УЧЕНИКИ. Сложность квадратичная $O(N \times M)$.
2. Merge Join + Sort → таблицы сортируются по ключам соединения, затем сливаются. Требуется дополнительной сортировки всех строк, что дорого без индексов.
3. Hash Join → строится хеш-таблица по ключу соединения, затем происходит пробег по другой таблице.

Оптимальный план: Hash Join.

Почему: При отсутствии индексов PostgreSQL выбирает алгоритм с линейной сложностью $O(N+M)$. Планировщик сначала выполнит Seq Scan по Н_ОБУЧЕНИЯ, отфильтрует НЗК = '001000', построит хеш по ЧЛВК_ИД, затем аналогично обработает Н_ЛЮДИ и Н_УЧЕНИКИ. Это минимизирует случайные чтения и avoids накладные расходы на сортировку.

EXPLAIN ANALYZE

```
Hash Join (cost=296.80..1158.38 rows=5 width=24) (actual time=2.483..8.285 rows=1 loops=1)
" Hash Cond: ("Н_УЧЕНИКИ"."ЧЛВК_ИД" = "Н_ЛЮДИ"."ИД")
"   -> Seq Scan on "Н_УЧЕНИКИ" (cost=0.00..774.11 rows=23311 width=8) (actual time=0.020..2.125 rows=23312 loops=1)
"   -> Hash (cost=296.79..296.79 rows=1 width=24) (actual time=2.367..2.376 rows=1 loops=1)
"         Buckets: 1024 Batches: 1 Memory Usage: 9kB
"         -> Hash Join (cost=119.78..296.79 rows=1 width=24) (actual time=0.589..2.371 rows=1 loops=1)
"               Hash Cond: ("Н_ЛЮДИ"."ИД" = "Н_ОБУЧЕНИЯ"."ЧЛВК_ИД")
"               -> Seq Scan on "Н_ЛЮДИ" (cost=0.00..163.97 rows=3474 width=20) (actual time=0.014..1.534 rows=3500 loops=1)
"                     Filter: (("ФАМИЛИЯ")::text > 'Иванов'::text)
"                     Rows Removed by Filter: 1619
"                     -> Hash (cost=119.76..119.76 rows=1 width=4) (actual time=0.518..0.522 rows=2 loops=1)
"                           Buckets: 1024 Batches: 1 Memory Usage: 9kB
"                           -> Seq Scan on "Н_ОБУЧЕНИЯ" (cost=0.00..119.76 rows=1 width=4) (actual time=0.096..0.513 rows=2 loops=1)
"                                 Filter: (("НЗК")::text = '001000'::text)
"                                 Rows Removed by Filter: 5020
Planning Time: 0.238 ms
Execution Time: 8.368 ms
```

Изменения при добавлении индексов

- Seq Scan по Н_ОБУЧЕНИЯ → Index Scan по ("НЗК", "ЧЛВК_ИД").
Фильтр и ключ соединения берутся сразу из индекса.
- Seq Scan по Н_ЛЮДИ → Index Scan по ("ФАМИЛИЯ", "ИД").
Диапазон проходится в дереве, ИД отдаётся без захода в таблицу.
- Hash Join → Nested Loop. После фильтрации строк становится мало, строить хеш невыгодно. Оптимизатор переходит на построчный поиск по индексам.
- ГРУППА возвращается через Index Only Scan по Н_УЧЕНИКИ. Общая стоимость падает в 10–50 раз.

EXPLAIN ANALYZE с индексами

```
Nested Loop (cost=0.85..127.86 rows=5 width=24) (actual time=0.120..0.351 rows=1 loops=1)
  -> Nested Loop (cost=0.56..127.35 rows=1 width=24) (actual time=0.116..0.345 rows=1 loops=1)
    "
    -> Index Only Scan using idx_q1_training_nzk_member_id on ""Н_ОБУЧЕНИЯ"" (cost=0.28..8.30 rows=1 width=4) (actual time=0.094..0.345 rows=1 loops=1)
        Index Cond: (""НЗК"" = '001000'::text)
        Heap Fetches: 1
    "
    -> Index Only Scan using idx_q1_people_surname_id on ""Н_ЛЮДИ"" (cost=0.28..119.04 rows=1 width=20) (actual time=0.119..0.345 rows=1 loops=1)
        Index Cond: (("ФАМИЛИЯ" > 'Иванов'::text) AND (""ИД"" = ""Н_ОБУЧЕНИЯ"". ""ЧЛВК_ИД""))
        Heap Fetches: 1
    "
  -> Index Only Scan using idx_q1_students_member_id_group on ""Н_УЧЕНИКИ"" (cost=0.29..0.46 rows=5 width=8) (actual time=0.004..0.004 rows=5 loops=1)
      Index Cond: (""ЧЛВК_ИД"" = ""Н_ЛЮДИ"". ""ИД"")
      Heap Fetches: 1
Planning Time: 0.453 ms
Execution Time: 0.418 ms
```

Доп задание

Таблицы

```
CREATE TABLE users (  
  
    user_id SERIAL PRIMARY KEY,  
  
    email VARCHAR(255) NOT NULL,  
  
    status VARCHAR(20) NOT NULL CHECK (status IN ('active', 'inactive',  
'suspended')),  
  
    region_code VARCHAR(10) NOT NULL,  
  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
  
);
```

```
CREATE TABLE orders (  
  
    order_id SERIAL PRIMARY KEY,  
  
    user_id INTEGER NOT NULL REFERENCES users(user_id),  
  
    order_date DATE NOT NULL,  
  
    total_amount DECIMAL(10,2) NOT NULL,  
  
    status VARCHAR(20) NOT NULL,  
  
    shipping_country VARCHAR(50) NOT NULL  
  
);
```

```
CREATE TABLE order_items (  
  
    item_id SERIAL PRIMARY KEY,  
  
    order_id INTEGER NOT NULL REFERENCES orders(order_id),
```

```
product_id INTEGER NOT NULL,  
  
quantity INTEGER NOT NULL,  
  
price_at_purchase DECIMAL(10,2) NOT NULL  
);
```

Запрос

```
SELECT * FROM users  
  
INNER JOIN orders o on users.user_id = o.user_id  
  
INNER JOIN order_items oi on o.order_id = oi.order_id  
  
WHERE o.total_amount > 900  
  
AND users.user_id > 340  
  
AND users.status = 'active';
```

Индексы

Было создано 2 индекса - B-tree для orders.amount, так как идет сортировка строгим сравнением существенного объема данных, а значит логарифмическая сложность сильно ускорит использование

А также users.status - Hash, так как статус хранится в строке, выбирается из существенного объема данных, а значит $O(1)$ значительно ускорит использование

```
CREATE INDEX idx_amount ON "orders" (total_amount);  
  
CREATE INDEX idx_user ON "users" USING HASH(status);
```

Было

```
Hash Join (cost=293.10..668.48 rows=272 width=103) (actual time=2.230..3.991 rows=272 loops=1)
  Hash Cond: (users.user_id = o.user_id)
  -> Seq Scan on users (cost=0.00..330.50 rows=8434 width=51) (actual time=0.012..1.246 rows=8446 loops=1)
    Filter: ((user_id > 340) AND ((status)::text = 'active'::text))
    Rows Removed by Filter: 4054
  -> Hash (cost=288.07..288.07 rows=402 width=52) (actual time=2.211..2.213 rows=402 loops=1)
    Buckets: 1024 Batches: 1 Memory Usage: 43kB
    -> Hash Join (cost=211.57..288.07 rows=402 width=52) (actual time=1.540..2.145 rows=402 loops=1)
      Hash Cond: (oi.order_id = o.order_id)
      -> Seq Scan on order_items oi (cost=0.00..66.00 rows=4000 width=22) (actual time=0.002..0.234 rows=4000 loops=1)
      -> Hash (cost=199.00..199.00 rows=1006 width=30) (actual time=1.534..1.535 rows=1010 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 71kB
        -> Seq Scan on orders o (cost=0.00..199.00 rows=1006 width=30) (actual time=0.002..1.405 rows=1010 loops=1)
          Filter: (total_amount > '900'::numeric)
          Rows Removed by Filter: 8990
Planning Time: 0.199 ms
Execution Time: 4.016 ms
```

Стало

```
Hash Join (cost=204.76..580.14 rows=272 width=103) (actual time=0.665..1.639 rows=272 loops=1)
  Hash Cond: (users.user_id = o.user_id)
  -> Seq Scan on users (cost=0.00..330.50 rows=8434 width=51) (actual time=0.008..0.672 rows=8446 loops=1)
    Filter: ((user_id > 340) AND ((status)::text = 'active'::text))
    Rows Removed by Filter: 4054
  -> Hash (cost=199.73..199.73 rows=402 width=52) (actual time=0.647..0.648 rows=402 loops=1)
    Buckets: 1024 Batches: 1 Memory Usage: 43kB
    -> Hash Join (cost=123.23..199.73 rows=402 width=52) (actual time=0.235..0.599 rows=402 loops=1)
      Hash Cond: (oi.order_id = o.order_id)
      -> Seq Scan on order_items oi (cost=0.00..66.00 rows=4000 width=22) (actual time=0.002..0.146 rows=4000 loops=1)
      -> Hash (cost=110.66..110.66 rows=1006 width=30) (actual time=0.227..0.228 rows=1010 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 71kB
        -> Bitmap Heap Scan on orders o (cost=24.08..110.66 rows=1006 width=30) (actual time=0.057..0.168 rows=1010 loops=1)
          Recheck Cond: (total_amount > '900'::numeric)
          Heap Blocks: exact=74
          -> Bitmap Index Scan on idx_amount (cost=0.00..23.83 rows=1006 width=0) (actual time=0.050..0.050 rows=1010 loops=1)
            Index Cond: (total_amount > '900'::numeric)
Planning Time: 0.520 ms
Execution Time: 1.673 ms
```

Изменения

Индекс для users.status не был использован, поскольку в этой таблице также проверяется id, что делает индекс бессмысленным

Индекс для orders.amount был использован, что значительно сократило время выполнения. Seq scan -> Bitmap Heap Scan/Bitmap index scan

Общее время выполнения уменьшено с 4ms до 1.6ms

